# Порождающие паттерны. Строитель

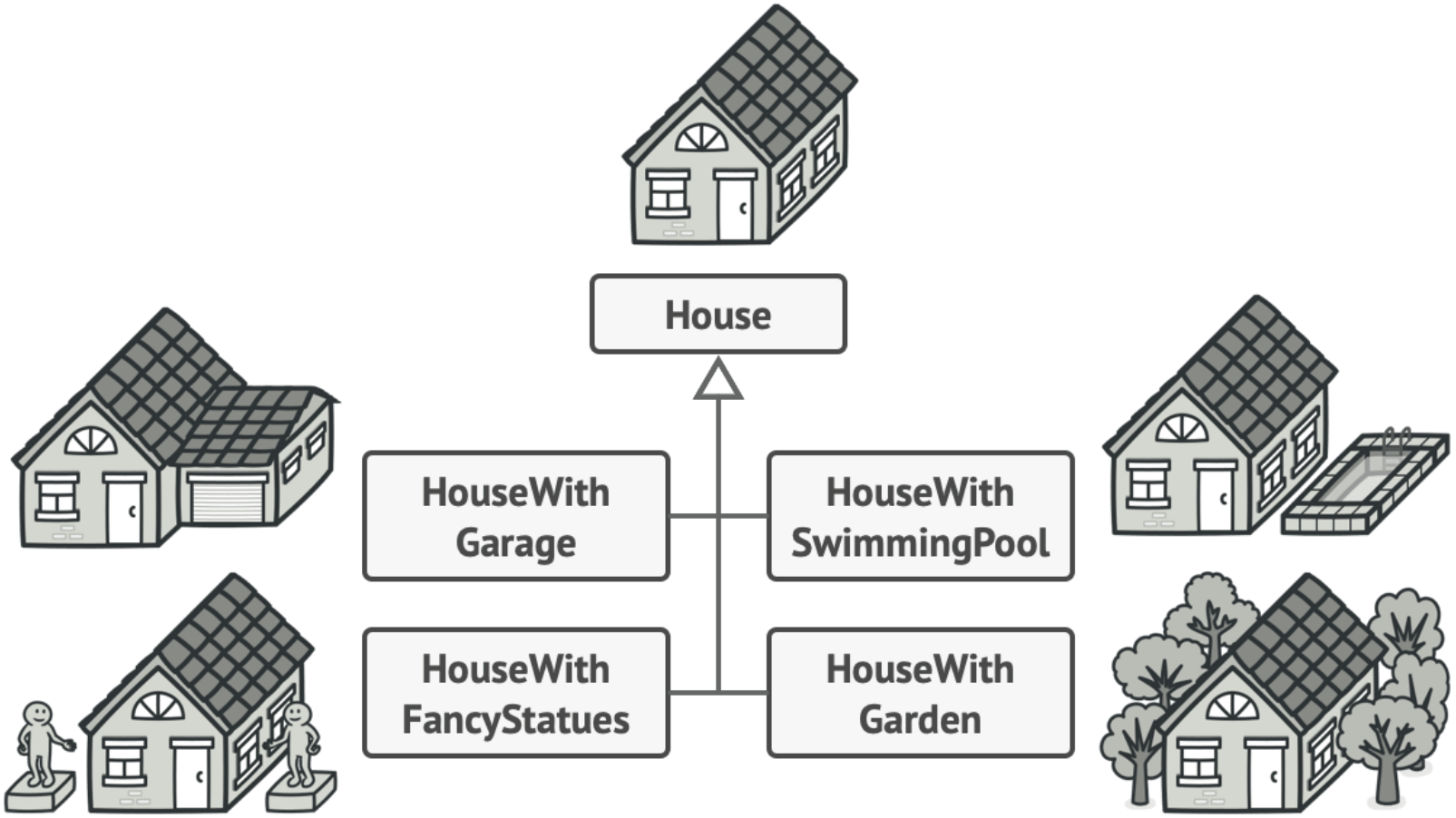
# Паттерн Строитель
# Builder



**Паттерн Builder** – помогает организовать пошаговое построение сложного объекта-продукта так, что клиенту не требуется понимать последовательность шагов и внутреннее устройство строящегося объекта-продукта, при этом в результате одного и того же процесса конструирования могут получаться объекты-продукты с различным представлением (внутренним устройством).

# Задача – получить разные представления (устройство) объектов

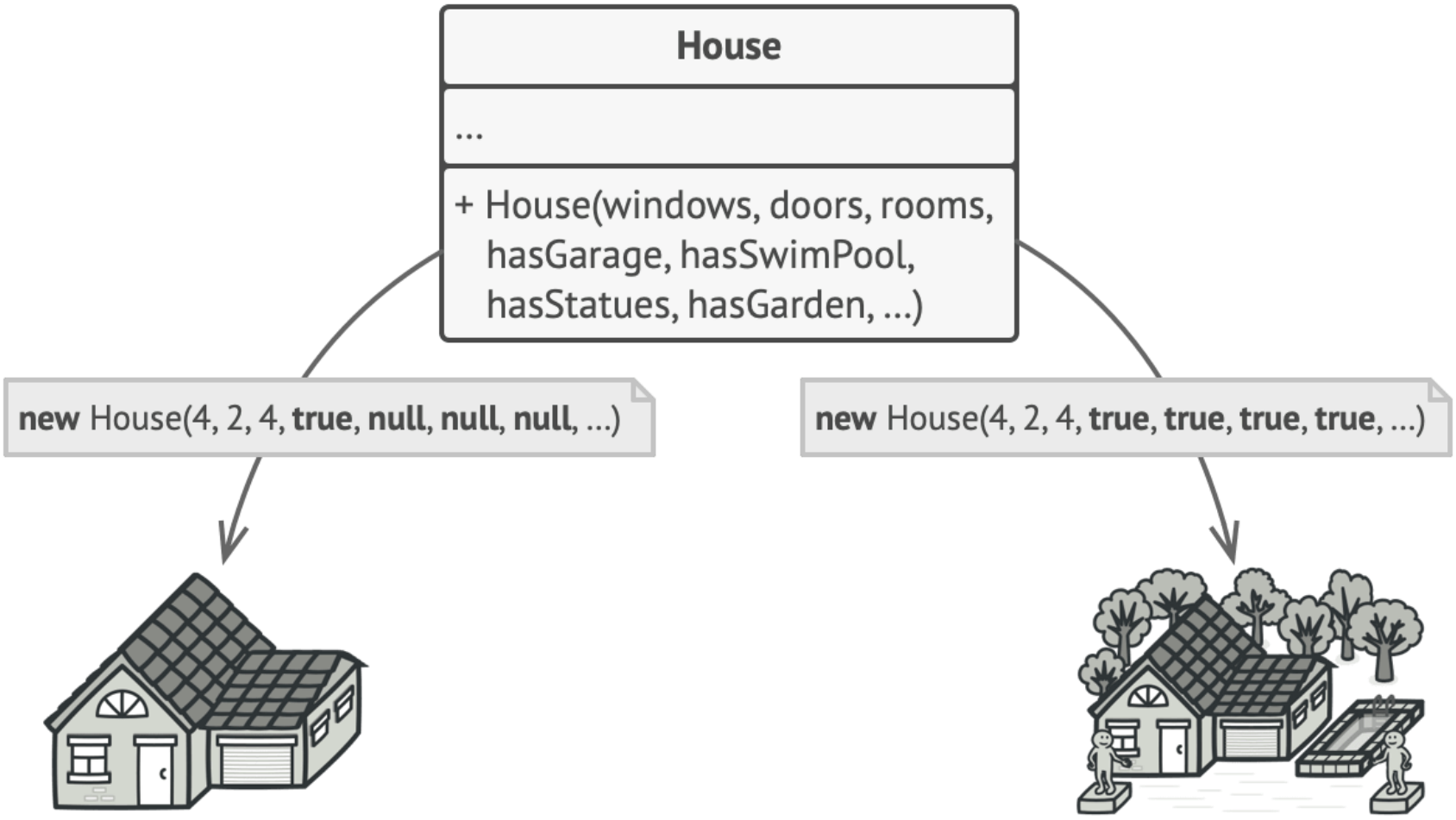
## Проблема 1 – сложная иерархия наследования




*Разрастание иерархии наследования, увеличение количества объектов (зависит от набора реализуемых вариантов*

# Задача – получить разные представления (устройство) объектов

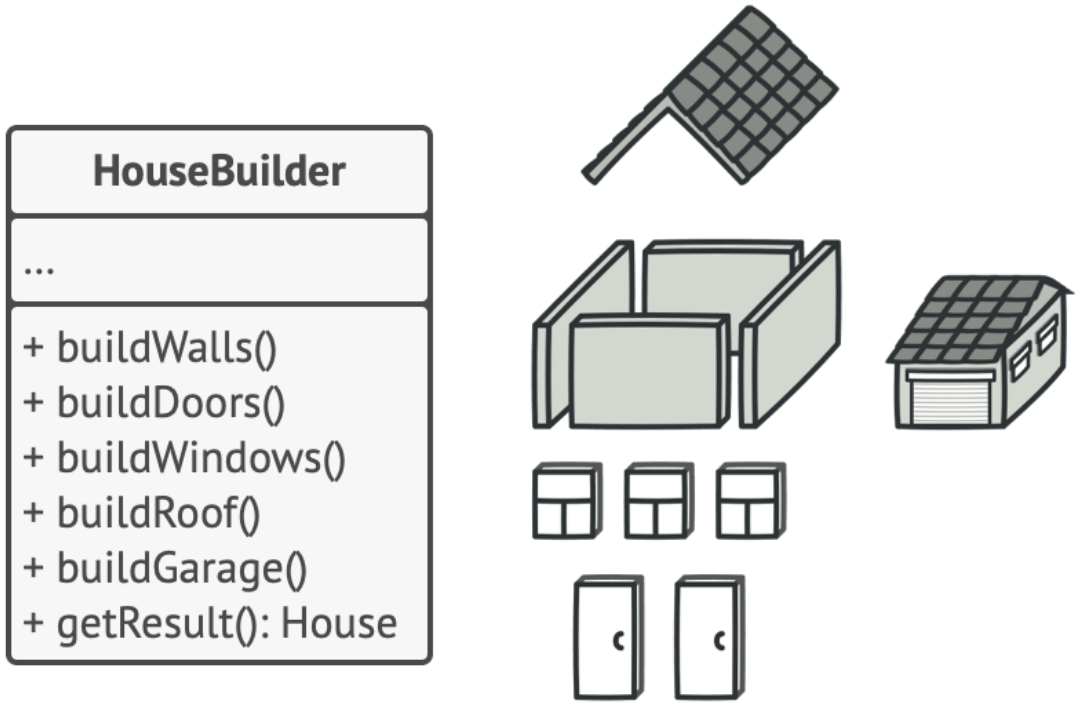
**Проблема 2 – конструктор, перегруженный (**не технология**) параметрами**




*Конструктор со множеством параметров имеет недостаток: не все параметры нужны большую часть времени.*

# Решение

Паттерн **Строитель** предлагает вынести конструирование объекта за пределы его собственного класса, поручив это дело отдельным объектам, которые следует называть *строителями*.




Паттерн предлагает разбить процесс конструирования объекта на отдельные шаги (*построитьСтены*, *вставитьДвери*..).

Чтобы создать объект, нужно поочерёдно вызывать методы строителя.

Не нужно запускать все шаги, а только те, которые необходимы для производства объекта определённой конфигурации.

# Решение

Можно создать несколько классов *строителей*, выполняющих одни и те же шаги по-разному (*строит из камня, дерева* …).

Используя этих строителей в одном и том же строительном процессе, можно получать на выходе объекты с различными характеристиками.

Рекомендуется выделить вызовы методов строителя в отдельный класс, называемый ***директором*** (прораб, бригадир). В этом случае ***директор*** будет *задавать порядок* шагов строительства, а ***строитель*** — *выполнять* их.

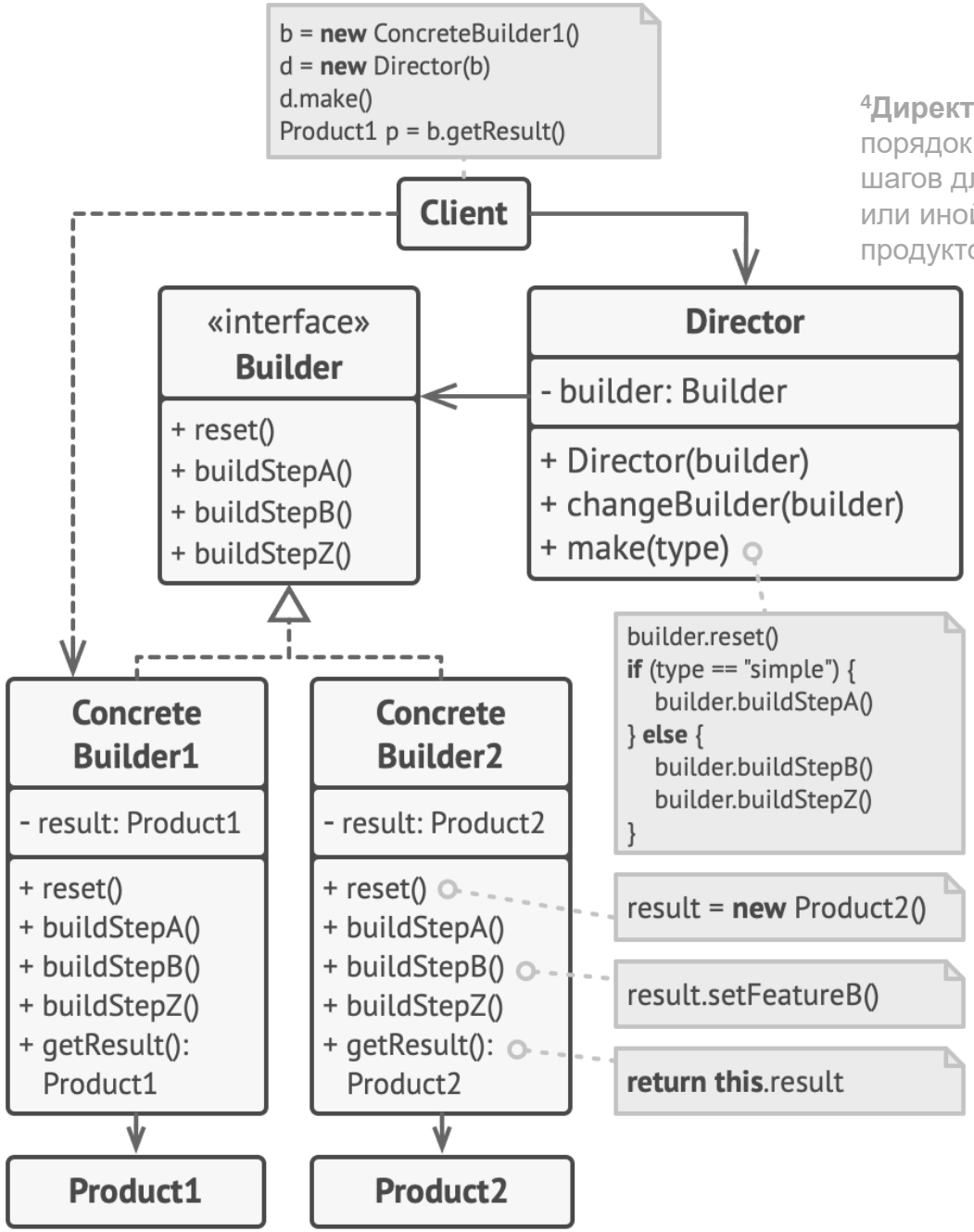
- Отдельный класс директора не является строго обязательным. Допускается вызывать методы строителя напрямую из клиентского кода.
- *Директор полезен, если есть несколько способов конструирования продуктов, отличающихся порядком и наличием шагов конструирования. В этом случае можно объединить всю эту логику в одном классе.*
- Клиенту необходимо привязать желаемого *строителя* к *директору*, а затем получить у строителя готовый результат.

# Структура паттерна

[1] **Интерфейс строителя** объявляет шаги конструирования продуктов, общие для всех видов строителей.

[2] **Конкретные строители** реализуют строительные шаги, каждый по-своему. Конкретные строители могут производить разнородные объекты, не имеющие общего интерфейса.

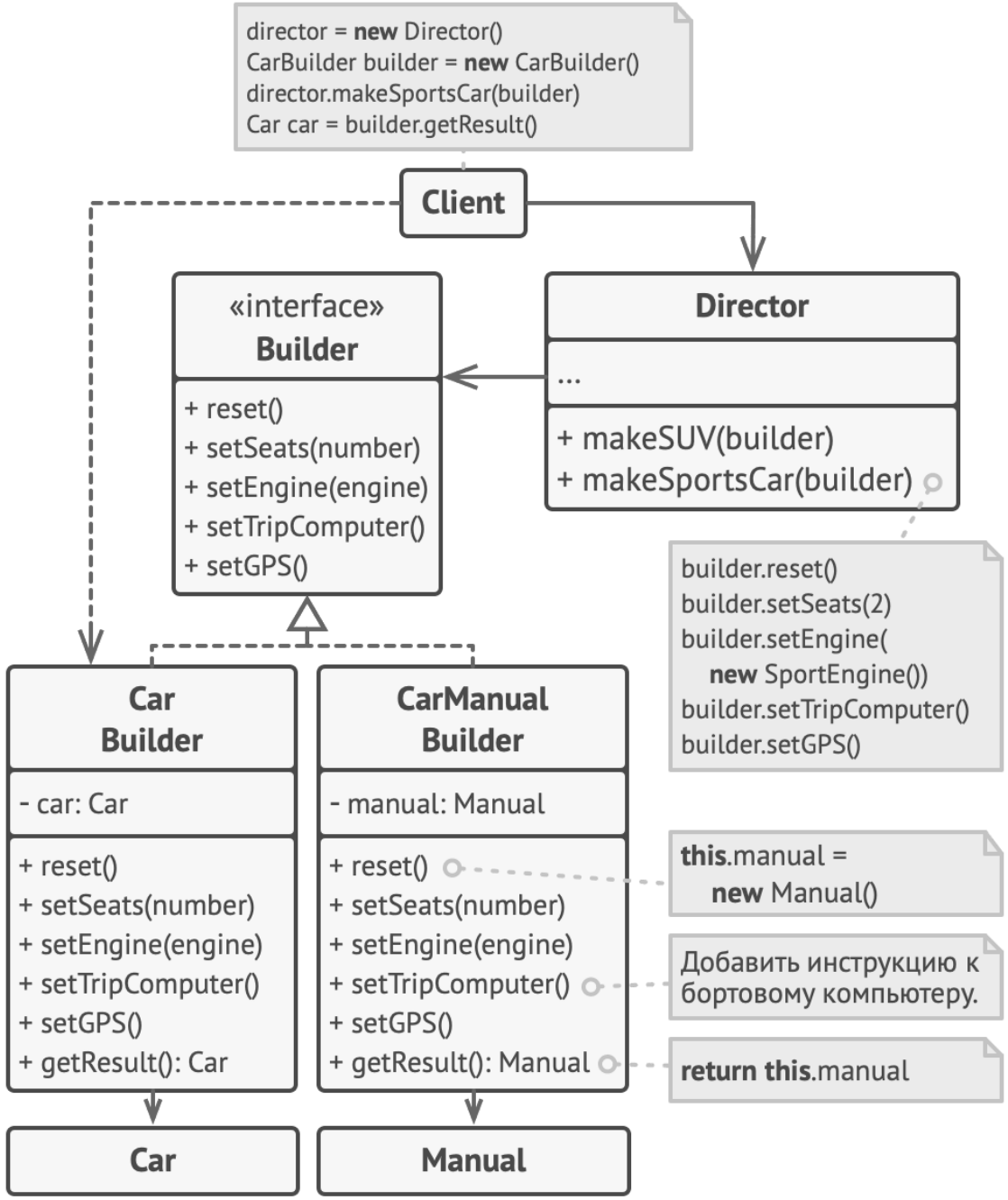
[3] **Продукт** — создаваемый объект. Продукты, сделанные разными строителями, не обязаны иметь общий интерфейс.




[4] **Директор** определяет порядок вызова строительных шагов для производства той или иной конфигурации продуктов.

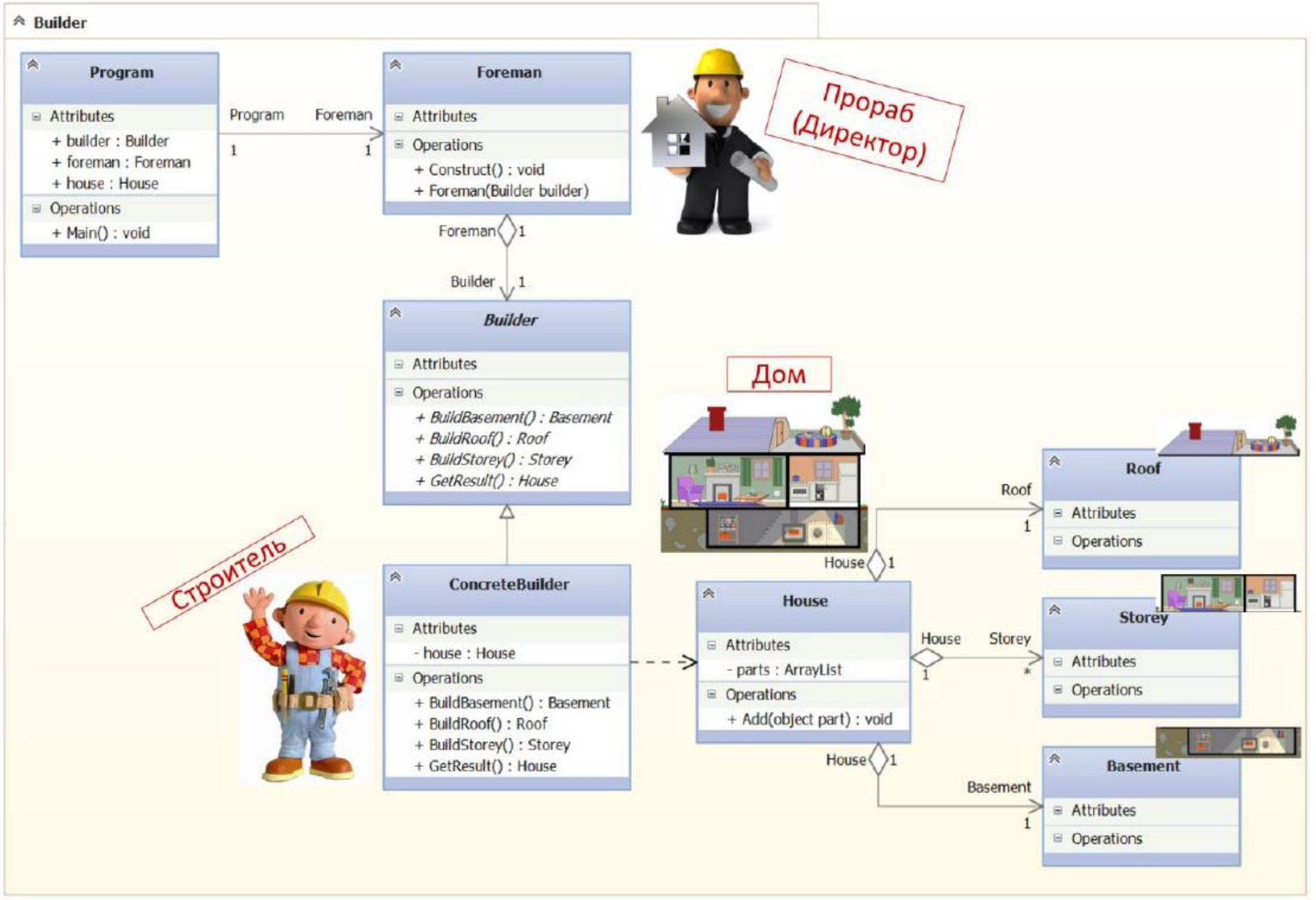
[5] Обычно **Клиент** подаёт в конструктор директора уже готовый объект-строитель, и в дальнейшем данный директор использует только его. Но возможен и другой вариант, когда клиент передаёт строителя через параметр строительного метода директора. В этом случае можно каждый раз применять разных строителей для производства различных представлений объектов.

# Пример: Строитель автомобилей и документации к ним

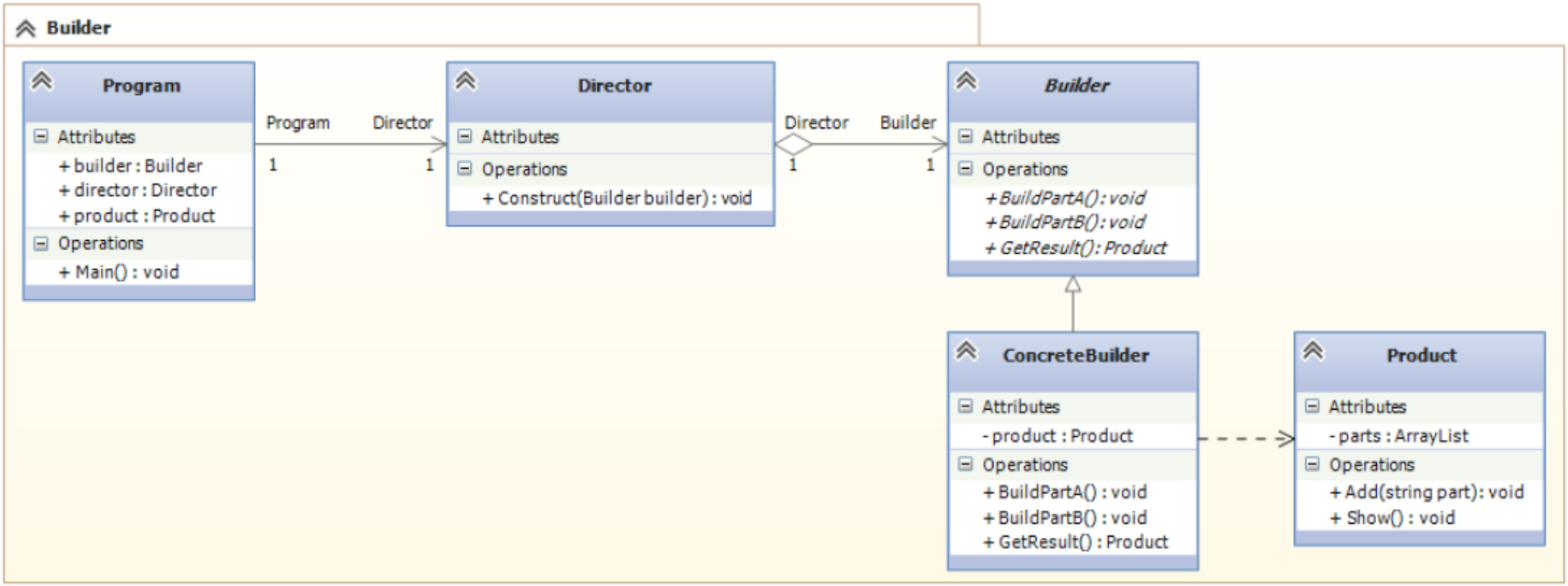



Пошаговое конструирование автомобилей и инструкций к ним

# Еще Пример


Builder
Program
Attributes
+ builder : Builder
+ foreman : Foreman
+ house : House
Operations
+ Main() : void
Program
1
Foreman
1
Foreman
Attributes
Operations
+ Construct() : void
+ Foreman(Builder builder)
Прораб (Директор)
Foreman
1
Builder
1
Builder
Attributes
Operations
+ BuildBasement() : Basement
+ BuildRoof() : Roof
+ BuildStorey() : Storey
+ GetResult() : House
Дом
Roof
1
Roof
Attributes
Operations
House
1
Строитель
ConcreteBuilder
Attributes
- house : House
Operations
+ BuildBasement() : Basement
+ BuildRoof() : Roof
+ BuildStorey() : Storey
+ GetResult() : House
House
Attributes
- parts : ArrayList
Operations
+ Add(object part) : void
House
1
Storey
*
Storey
Attributes
Operations
House
1
Basement
1
Basement
Attributes
Operations

# Структура паттерна на языке UML



# Структура паттерна на языке C#

```
class Program
{
    public static void Main()
    {
        Builder builder = new ConcreteBuilder();
        Director director = new Director(builder);
        director.Construct();
        Product product = builder.GetResult();
        product.Show();
    }
}
```

```
class Director
{
    Builder builder;

    public Director(Builder builder)
    {
        this.builder = builder;
    }

    public void Construct()
    {
        builder.BuildPartA();
        builder.BuildPartB();
        builder.BuildPartC();
    }
}
```

```
abstract class Builder
{
    public abstract void BuildPartA();
    public abstract void BuildPartB();
    public abstract void BuildPartC();
    public abstract Product GetResult();
}
```

```
class ConcreteBuilder : Builder
{
    Product product = new Product();

    public override void BuildPartA()
    {
        product.Add("Part A");
    }

    public override void BuildPartB()
    {
        product.Add("Part B");
    }

    public override void BuildPartC()
    {
        product.Add("Part C");
    }

    public override Product GetResult()
    {
        return product;
    }
}
```

```
class Product
{
    ArrayList parts = new ArrayList();

    public void Add(string part)
    {
        parts.Add(part);
    }

    public void Show()
    {
        foreach (string part in parts)
            Console.WriteLine(part);
    }
}
```

## Участники

Product - **Продукт**:
Представляет собой класс сложно-конструируемого объекта-продукта и содержит в себе набор методов для сборки конечного результата-продукта из частей. Класс продукта может быть связан связями отношений агрегации, с классами которые описывают составные части создаваемого продукта.

Builder - **Абстрактный строитель**:
Предоставляет набор абстрактных методов (интерфейс) для создания объекта-продукта из частей и получения готового результата.

ConcreteBuilder - **Конкретный строитель**:
Конструирует объект-продукт собирая его из частей, реализуя интерфейс, заданный абстрактным строителем (Builder). Предоставляет доступ к готовому продукту (возвращает продукт клиенту или в частном случае директору (Director)).

Director – **Директор (Распорядитель)**:
Пользуясь интерфейсом строителя (Builder), директор дает строителю указание построить продукт.

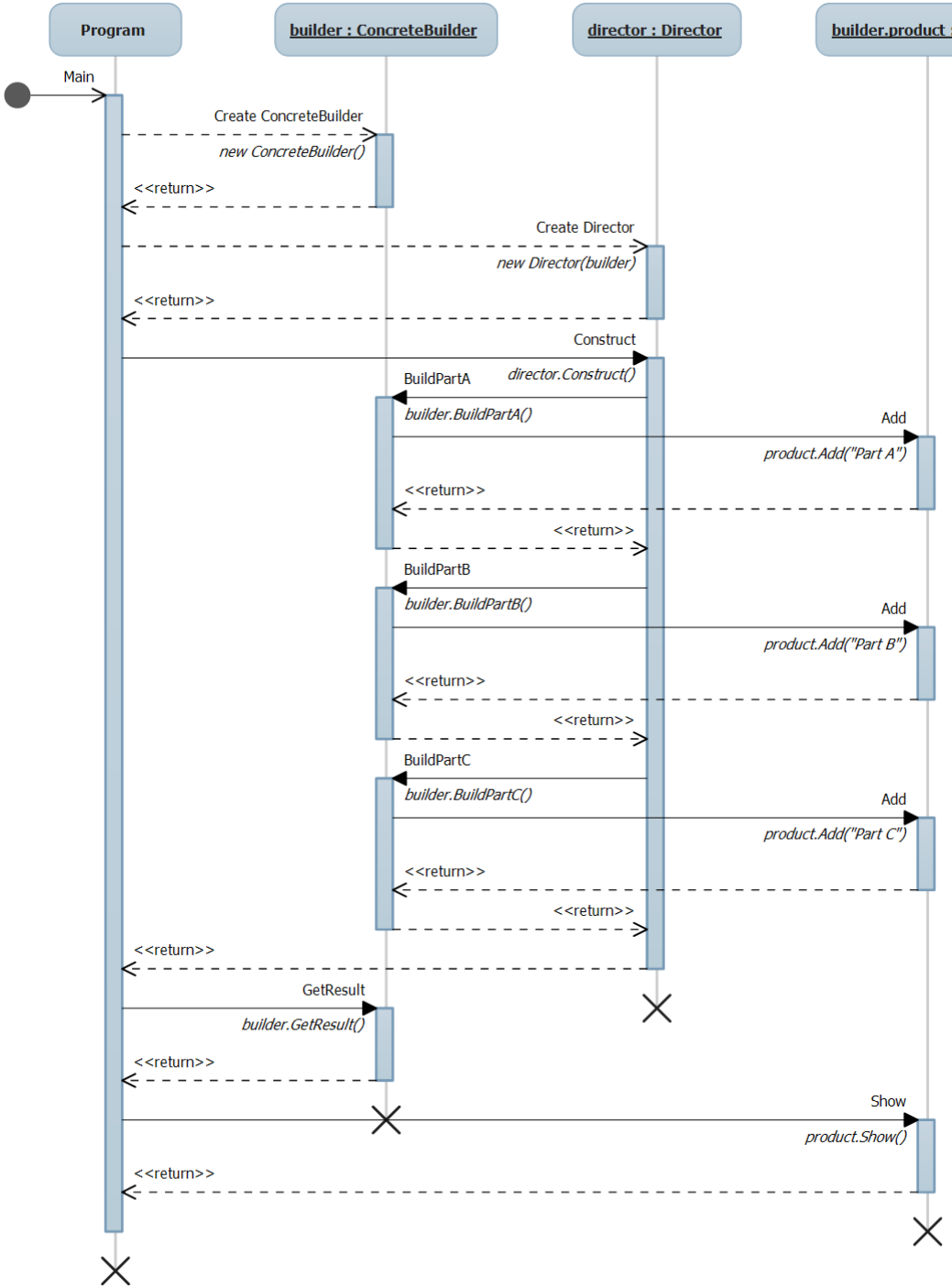
## Отношения между участниками

### Отношения между классами

- Класс Director имеет отношение *агрегации* с абстрактным классом Builder.
- Класс ConcreteBuilder имеет отношения *наследования* с абстрактным классом Builder и отношения *зависимости* с классом Product.
- Класс Product может иметь  отношения *агрегации* с классами частей (Part).

### Отношения между объектами

- ***Клиент*** создает экземпляр класса ConcreteBuilder.
- ***Клиент*** создает экземпляр класса Director при этом в качестве аргумента конструктора передает ссылку на ранее созданный экземпляр класса ConcreteBuilder.
- ***Директор*** (Director) вызывает на строителе (ConcreteBuilder) методы, тем самым уведомляя строителя о том, что требуется построить определенную часть продукта.
- ***Строитель*** выполняет операции по построению продукта, добавляя к продукту те части, которые указывает директор (Director).
- ***Клиент*** получает от строителя ссылку на экземпляр построенного продукта.




Отношения между объектами (директором и строителем).

**1. Если нужно избавиться от *«телескопического конструктора»:***

```
class Pizza {
    Pizza(int size) { ... }
    Pizza(int size, boolean cheese) { ... }
    Pizza(int size, boolean cheese, boolean pepperoni) { ... }
    // ...
```

Паттерн ***Строитель*** позволяет собирать объекты пошагово, вызывая только те шаги, которые нужны.

**2. Когда код должен создавать разные представления какого-то объекта.**

***Строитель*** можно применить, если создание нескольких представлений объекта состоит из одинаковых этапов, которые отличаются в деталях.

Интерфейс ***строителей*** определит все возможные этапы конструирования. Каждому представлению будет соответствовать собственный ***класс-строитель***. А порядок этапов строительства будет задавать ***класс-директор***.

**3. Когда нужно собирать сложные составные объекты**

Строитель конструирует объекты пошагово, а не за один проход. Шаги строительства можно выполнять рекурсивно.

***Строитель*** не позволяет посторонним объектам иметь доступ к конструируемому объекту, пока тот не будет полностью готов. Это предохраняет клиентский код от получения незаконченных «битых» объектов.

# Шаги реализации

1. **Проверить**, что создание разных представлений объекта можно свести к общим шагам.
2. **Описать** эти шаги в общем интерфейсе строителей.
3. **Для каждого из представлений *объекта-продукта*** создать по одному *классу-строителю* и реализовать их методы строительства. Важно: предусмотреть метод получения результата. Обычно конкретные строители определяют собственные методы получения результата строительства.
4. **Проанализировать необходимость создании класса директора**. Его методы будут создавать различные конфигурации продуктов, вызывая разные шаги одного и того же строителя.
5. **Клиентский код должен будет создавать и объекты строителей, и объект директора**. Перед началом строительства клиент должен связать определённого строителя с директором. Это можно сделать либо через конструктор, либо через *сеттер*, либо, подав строителя напрямую в строительный метод директора.
6. **Результат строительства можно вернуть из директора**, но только если метод возврата продукта удалось поместить в общий интерфейс строителей. Иначе директор будет жёстко привязан к конкретным классам строителей.

# Преимущества и недостатки

+ Позволяет создавать продукты пошагово.

+ Позволяет использовать один и тот же код для создания различных продуктов.

+ Изолирует сложный код сборки продукта от его основной бизнес-логики (логика предметной области).

- Усложняет код программы из-за введения дополнительных классов.

- Клиент будет привязан к конкретным классам строителей, так как в интерфейсе директора может не быть метода получения результата.